GeoWombat: Scalable geospatial and remote sensing analysis in Python

7 April 2026

Summary

GeoWombat is an open-source Python library that provides an end-to-end platform for geospatial raster data processing and remote sensing analysis at scale. Built on xarray (Hoyer and Hamman 2017), Dask (Rocklin 2015), and rasterio (Gillies et al. 2013--), GeoWombat simplifies common but complex operations - such as mosaicking multi-tile imagery, reprojecting across coordinate reference systems, aligning rasters of varying resolutions, and performing radiometric corrections - into concise, intuitive commands. The library includes built-in sensor profiles for Landsat 1–8, Sentinel-1 and Sentinel-2, MODIS, and NAIP that automate band naming, scaling, and metadata handling. It supports workflows spanning cloud-based data access via SpatioTemporal Asset Catalogs (STAC) from multiple providers, a full radiometric processing chain (DN-to-reflectance conversion, atmospheric correction, BRDF normalization, and topographic correction), vegetation index computation, raster-vector interoperability, Cython-accelerated moving window statistics, scikit-learn-based (Pedregosa et al. 2011) machine learning classification, and deep learning with PyTorch (Paszke et al. 2019). By leveraging Dask's lazy evaluation and task graphs, GeoWombat enables out-of-core processing of raster datasets of any size on commodity hardware, with optional distributed computing via Ray.

Statement of need

Satellite remote sensing has become essential for environmental monitoring, agriculture, disaster management, and land use analysis. Modern sensor constellations such as Sentinel-2, Landsat, and PlanetScope produce petabytes of freely available imagery, yet the Python ecosystem for processing this data remains fragmented. A typical analysis workflow requires stitching together multiple libraries, rasterio for I/O, GDAL (GDAL/OGR contributors 2024) for warping, NumPy (Harris et al. 2020) for array math, xarray for labeled dimensions, geopandas (Jordahl et al. 2020) for vector operations, and scikit-learn for modeling, each with its own conventions for handling coordinate reference systems, nodata values, affine transforms, and chunked computation.

Even straightforward tasks such as mosaicking two adjacent Landsat tiles require the analyst to understand affine transformations, coordinate reference system alignment, and resampling strategies. More complex workflows, multi-temporal classification, BRDF-adjusted surface reflectance, or continental-scale time series analysis, demand substantial boilerplate code and deep expertise in the underlying data structures. This complexity creates a barrier for domain scientists (ecologists, agronomists, urban planners) who need to work with satellite imagery but are not geospatial

software engineers.

GeoWombat addresses this gap by providing a unified, high-level API that orchestrates these lower-level libraries behind a consistent interface. Its target audience includes GIS professionals, remote sensing scientists, and machine learning practitioners who need to process, analyze, and model large-scale raster data without writing low-level geospatial code.

State of the field

Several Python packages address parts of the geospatial raster analysis workflow. Rioxarray (Snow et al. 2022) extends xarray with rasterio-backed I/O and CRS-aware operations but does not provide remote sensing-specific functionality such as vegetation indices, QA masking, or classification pipelines. Google Earth Engine (Gorelick et al. 2017) offers cloud-based processing of global archives but requires an internet connection, operates within a proprietary platform, and limits user control over computation. Xee (Google 2023) bridges Earth Engine with xarray but inherits the same platform constraints. Open Data Cube (Killough 2018) provides a database-indexed approach to analysis-ready data but requires infrastructure setup and data ingestion. Raster Vision (Azavea 2023) focuses specifically on deep learning for geospatial imagery and does not cover the broader analysis workflow.

GeoWombat differentiates itself by offering a comprehensive, local-first toolkit that spans the full analysis chain, from data access through modeling, within a single API. Its context manager pattern for on-the-fly reprojection and alignment, built-in sensor profiles for automatic band naming and scaling, and tight integration with scikit-learn pipelines and PyTorch models provide a uniquely cohesive workflow that the above tools do not individually replicate.

Software design

GeoWombat's architecture centers on three design principles: (1) lazy evaluation for scalability, (2) configuration-driven defaults for usability, and (3) xarray accessor extensibility.

Lazy evaluation. All raster operations return Dask-backed xarray DataArrays. Computation is deferred until the user explicitly calls `.compute()` or `.gw.save()`, allowing GeoWombat to build optimized task graphs over arbitrarily large datasets. Chunk sizes are configurable and default to 512 x 512 pixels.

Configuration context manager. The `gw.config.update()` context manager sets global parameters, sensor type, reference CRS, spatial resolution, bounding box, and nodata handling, that propagate to all downstream operations:

```
import geowombat as gw

with gw.config.update(sensor='s2', ref_crs=4326):
    with gw.open(['tile1.tif', 'tile2.tif'],
                 mosaic=True, overlap='mean') as src:
        ndvi = src.gw.ndvi()
        ndvi.gw.save('ndvi_mosaic.tif')
```

Accessor pattern. Geospatial methods are attached to xarray DataArrays via the `.gw` accessor

(`src.gw.ndvi()`, `src.gw.save()`, `src.gw.extract()`), keeping the namespace clean while providing discoverability.

The library is organized into the following modules:

- **I/O and backends** (`backends/`): Rasterio and GDAL-backed reading and writing with on-the-fly warping and mosaicking. Output is supported in GeoTIFF, NetCDF, VRT, and Zarr formats via `.gw.to_raster()`, `.gw.to_netcdf()`, `.gw.to_vrt()`, and `.gw.to_zarr()`.
- **STAC and cloud data access** (`core/stac.py`): Functions for searching, opening, compositing, and merging imagery from STAC catalogs. Multiple providers are supported including Element84, Microsoft Planetary Computer, and NASA LP CLOUD, with built-in collection definitions for Landsat Collection 2, Sentinel-1 GRD, Sentinel-2 L2A, HLS, NAIP, USDA Cropland Data Layer, and Copernicus DEM.
- **Radiometry** (`radiometry/`): A full radiometric processing chain including digital number to top-of-atmosphere reflectance and surface reflectance conversions, Dark Object Subtraction atmospheric correction, BRDF normalization via the global c-factor method (Roy et al. 2016), topographic corrections (slope, aspect, and illumination normalization from DEMs), 6S radiative transfer modeling, and Landsat/Sentinel pixel angle extraction. Quality assurance masking decodes sensor-specific bit-packed flags for Landsat, Sentinel-2 Scene Classification, and HLS Fmask.
- **Vegetation indices and band math** (`core/vi.py`): NDVI, EVI, EVI2, AVI, NBR, KNDVI, GCVI, a water index, generic normalized differences, and tasseled cap transformations (brightness, greenness, wetness) with sensor-specific coefficients.
- **Spatial operations** (`core/sops.py`): Point and polygon extraction, random and stratified sampling, subsetting, clipping, masking, value replacement, reclassification (`recode`), area calculation, and sub-pixel co-registration via AROSICS. Coordinate conversion utilities transform between pixel indices, map coordinates, and longitude/latitude. Raster-to-vector and vector-to-raster conversions enable interoperability with geopandas (Jordahl et al. 2020).
- **Moving window statistics** (`moving/`): Cython/OpenMP-accelerated focal operations including mean, standard deviation, and percentile calculations over configurable window sizes.
- **Machine learning** (`ml/`): Integration with scikit-learn pipelines for supervised and unsupervised classification and regression, including `fit()`, `predict()`, and `fit_predict()` workflows with built-in support for spatial cross-validation (Figure 1). Deep learning is supported via PyTorch (Paszke et al. 2019) with TorchGeo (Stewart et al. 2021) integration for architectures including TabNet, Lightweight Temporal Attention Encoder (L-TAE), and segmentation models (UNet, DeepLabV3).
- **Time series analysis** (`core/series.py`): The `gw.series()` interface processes multi-date image stacks with GPU acceleration via JAX (Bradbury et al. 2018), computing per-pixel temporal statistics including mean, median, amplitude, coefficient of variation, linear slope, percentiles, and quarterly decompositions.
- **Task workflows** (`tasks/`): A directed acyclic graph builder for defining and executing multi-step processing pipelines, with optional distributed execution via Ray.

```
labels_poly = gpd.read_file(training_labels.geojson)
```

```
pipe = Pipeline(
    [('scaler', StandardScaler()),
        ('pca', PCA(2)),
        ('clf', GaussianNB()),]
)

with gw.config.update(ref_res=150):
    with gw.open(l8_224078_20200518, nodata=0) as src:
        y = fit_predict(data=src, clf=pipe, labels=labels_poly, col='lc')
        y.plot(robust=True)
```

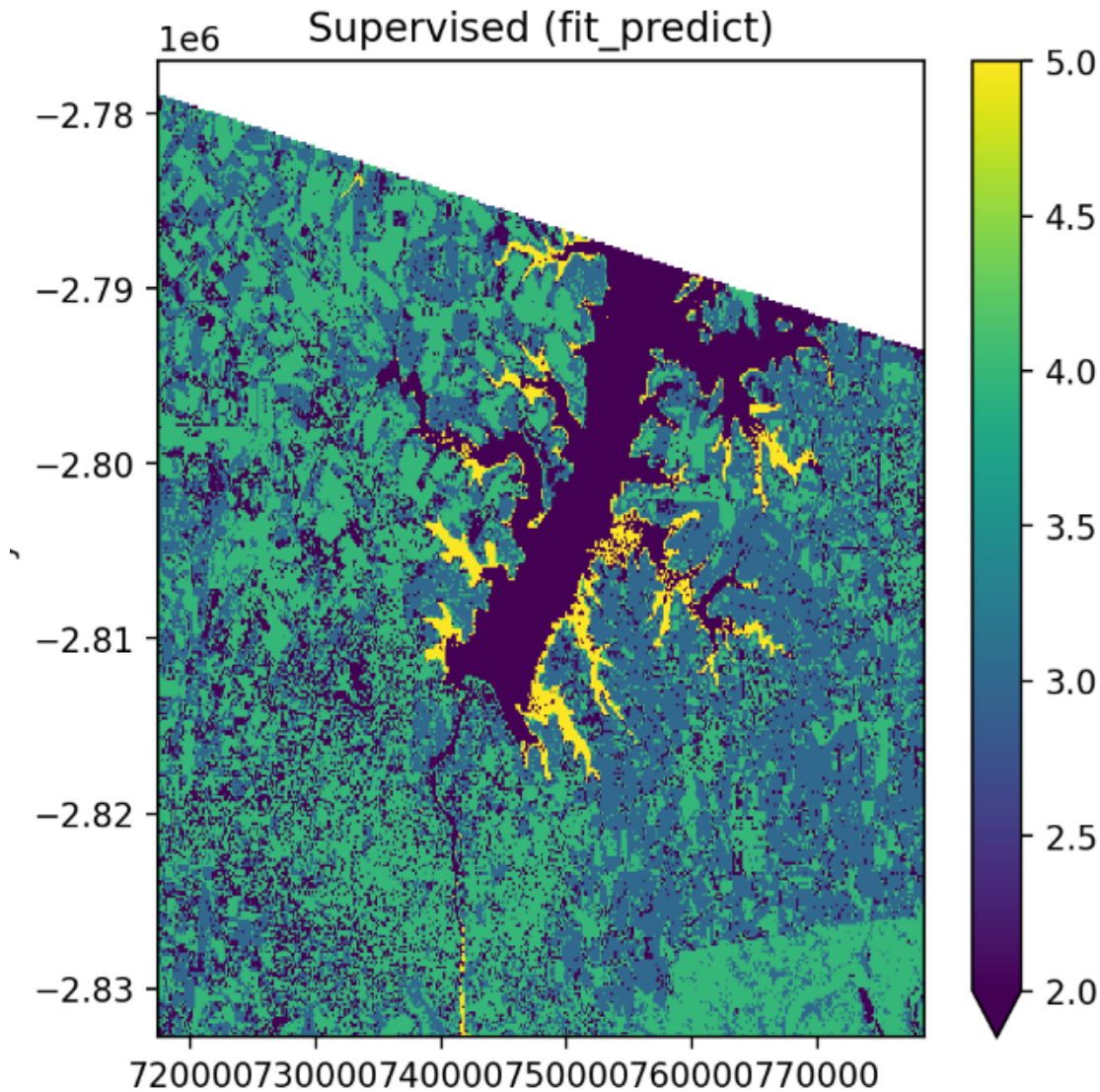

Figure 1: Multi-band land cover classification using GeoWombat's scikit-learn pipeline integration. The `fit_predict` method demonstrate the library's ability to train and predict ML piplines with minimal code.

Research impact

GeoWombat has been used in peer-reviewed research spanning land cover mapping, agricultural monitoring, and environmental change detection (M. Mann et al. 2023; Park 2021; Hernandez et al. 2024; Dorman et al. 2025; Hersh, Engstrom, and Mann 2021; Huang, Gao, and Foroutan 2025). It is actively used in graduate courses at The George Washington University for teaching remote sensing and geospatial machine learning through pygis.io (M. L. Mann 2025). The library's documentation includes tutorials covering all major features, and it is installable via pip and conda-forge, with over 50,000 downloads. The project maintains continuous integration testing across Python 3.10–3.12 on Linux, and welcomes contributions via its GitHub repository at https://github.com/jgrss/geowombat.

AI usage disclosure

The paper was initially drafted in full by the authors. Generative AI (Claude, Anthropic) was subsequently used to improve clarity, restructure content, and conform the manuscript to JOSS formatting requirements. All content was reviewed and verified by the authors. No generative AI was used in the initial five years of development of the GeoWombat software but is now being used to help maintain it and add new features.

Acknowledgements

We thank all contributors to the GeoWombat project, including Leonardo Hardtke, Robert Denham, and Sharon Xu. This project was not financially supported by any grant funding.

References

Azavea. 2023. *Raster Vision: Deep Learning for Aerial and Satellite Imagery*. https://github.com/azavea/raster-vision.

Bradbury, James, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, et al. 2018. *JAX: Composable Transformations of Python+NumPy Programs* (version 0.3.13). http://github.com/google/jax.

Dorman, Michael, Anita Graser, Jakub Nowosad, and Robin Lovelace. 2025. *Geocomputation with Python*. 1st ed. Chapman; Hall/CRC. https://doi.org/10.1201/9781003379911.

GDAL/OGR contributors. 2024. *GDAL/OGR Geospatial Data Abstraction Software Library*. Open Source Geospatial Foundation. https://doi.org/10.5281/zenodo.5884351.

Gillies, Sean et al. 2013--. *Rasterio: Geospatial Raster i/o for Python Programmers*. Mapbox. https://github.com/rasterio/rasterio.

Google. 2023. *Xee: An Xarray Extension for Google Earth Engine*. https://github.com/google/Xee.

Gorelick, Noel, Matt Hancher, Mike Dixon, Simon Ilyushchenko, David Thau, and Rebecca Moore. 2017. "Google Earth Engine: Planetary-Scale Geospatial Analysis for Everyone." *Remote Sensing of Environment* 202: 18–27. https://doi.org/10.1016/j.rse.2017.06.031.

Harris, Charles R., K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, et al. 2020. "Array Programming with NumPy." *Nature* 585 (7825): 357–62. https://doi.org/10.1038/s41586-020-2649-2.

Hernandez, Alexander, Shaun Bushman, Paul Johnson, Matthew D. Robbins, and Kaden Patten.

2024. "Prediction of Turfgrass Quality Using Multispectral UAV Imagery and Ordinal Forests: Validation Using a Fuzzy Approach." *Agronomy* 14 (11). https://doi.org/10.3390/agronomy14112575.

Hersh, Jonathan, Ryan Engstrom, and Michael Mann. 2021. "Open Data for Algorithms: Mapping Poverty in Belize Using Open Satellite Derived Features and Machine Learning." *Information Technology for Development* 27 (2): 263–92. https://doi.org/10.1080/02681102.2020.1811945.

Hoyer, Stephan, and Joe Hamman. 2017. "Xarray: N-D Labeled Arrays and Datasets in Python." *Journal of Open Research Software* 5 (1). https://doi.org/10.5334/jors.148.

Huang, Xinyue, Wenyu Gao, and Hosein Foroutan. 2025. "Impact of Topographic Wind Conditions on Dust Particle Size Distribution: Insights from a Regional Dust Reanalysis Dataset." *Atmospheric Chemistry and Physics* 25: 9583–9600. https://doi.org/10.5194/acp-25-9583-2025.

Jordahl, Kelsey, Joris Van den Bossche, Martin Fleischmann, Jacob Wasserman, James McBride, Jeffrey Gerard, Jeff Tratner, et al. 2020. *Geopandas/Geopandas: V0.8.1* (version v0.8.1). Zenodo. https://doi.org/10.5281/zenodo.3946761.

Killough, Brian. 2018. "Overview of the Open Data Cube Initiative." In *IGARSS 2018 - 2018 IEEE International Geoscience and Remote Sensing Symposium*, 8629–32. https://doi.org/10.1109/IGARSS.2018.8517694.

Mann, Michael L. 2025. "Xr_fresh: Automated Time Series Feature Extraction for Remote Sensing and Gridded Data." *Journal of Open Source Software* 10 (115): 9009. https://doi.org/10.21105/joss.09009.

Mann, Michael, Steven Chao, Christophe Van Neste, Roger Lew, and Mark Isken. 2023. *Mmann1123/pyGIS: Clearer Satellites* (version v1.2.1). Zenodo. https://doi.org/10.5281/zenodo.8215141.

Park, Michael L. AND Flint, Isaac W. AND Mann. 2021. "Relationships of Climate, Human Activity, and Fire History to Spatiotemporal Variation in Annual Fire Probability Across California." *PLOS ONE* 16 (11): 1–20. https://doi.org/10.1371/journal.pone.0254723.

Paszke, Adam, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, et al. 2019. "PyTorch: An Imperative Style, High-Performance Deep Learning Library." In *Advances in Neural Information Processing Systems*, edited by H. Wallach, H. Larochelle, A. Beygelzimer, F. dAlché-Buc, E. Fox, and R. Garnett. Vol. 32. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf.

Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, et al. 2011. "Scikit-Learn: Machine Learning in Python." *Journal of Machine Learning Research* 12 (85): 2825–30. http://jmlr.org/papers/v12/pedregosa11a.html.

Rocklin, Matthew. 2015. "Dask: Parallel Computation with Blocked Algorithms and Task Scheduling." In *Proceedings of the 14th Python in Science Conference*, edited by Kathryn Huff and James Bergstra, 130–36. https://doi.org/10.25080/Majora-7b98e3ed-013.

Roy, D. P., H. K. Zhang, J. Ju, J. L. Gomez-Dans, P. E. Lewis, C. B. Schaaf, Q. Sun, J. Li, H. Huang, and V. Kovalskyy. 2016. "A General Method to Normalize Landsat Reflectance Data to Nadir BRDF Adjusted Reflectance." *Remote Sensing of Environment* 176: 255–71. https://doi.org/10.1016/j.rse.2016.01.023.

Snow, Alan D. et al. 2022. *Rioxarray: Rasterio Xarray Extension*. https://github.com/corteva/rioxarray.

Stewart, Adam J., Caleb Robinson, Isaac A. Corley, Anthony Ortiz, Juan M. Lavista Ferres, and Arindam Banerjee. 2021. "TorchGeo: Deep Learning with Geospatial Data." *CoRR* abs/2111.08872. https://arxiv.org/abs/2111.08872.